

Monopoly Deal Cards Game

Phase 2 Final Project Report

Group 7 - Software Engineering Project

Submission Stage	Phase-2 Final Submission
Course Project	Software Engineering Project - Monopoly Deal Cards Game
GitHub Repository	https://github.com/NEVER-AGAIN-RAY/Monopoly_Group7
Main Technology Stack	Java 17, Maven, JUnit 5, Gson, Tyrus WebSocket, JavaFX + FXML

Report Roadmap

1 Project Overview and Functional Scope	2 Clean Design and Architecture	3 Core Feature Implementation	4 Innovation: DeepSeek, Local Models, and Online Play
5 Testing and Validation	6 Teamwork and Member Contributions		

1. Project Overview and Functional Scope

This project implements a digital card-game version of Monopoly Deal Cards Game. The project follows a complete software engineering process, including requirements analysis, UML modeling, object-oriented design, modular implementation, GUI presentation, version-control collaboration, and testing validation.

The system uses the Java backend as the center for rule and state management, while the JavaFX + FXML desktop client provides the main interactive interface. The client communicates with the server through WebSocket JSON. The server is responsible for creating sessions, maintaining decks and player states, validating the legality of card play, resolving rent and action-card effects, and synchronizing the corresponding game state to different players. The Web/Vite side is kept as an auxiliary entry, mainly for verifying online rooms, different seat combinations, and multi-scenario flows.

Clean Design The latest code splits large entry points into single-responsibility collaborators: ClientCommandHandler handles client commands, SnapshotBuilder assembles snapshots, SessionFactory initializes sessions, and SeatAssembler builds player/AI seats.	Functionality Supports 2-5 players, HVM/PVP, AI difficulty, drawing cards, three plays per turn, discarding, rent, action cards, Just Say No, victory detection, pause/resume, quit, wild-property recoloring, save/load, and multiplayer privacy synchronization; JavaFX is the main GUI.
Teamwork The repository uses GitHub distributed version control. The commit history covers requirements, UML, architecture, GUI, rules, testing, documentation, and final refactoring. Member contributions are counted across the whole project process, not only by a single commit count.	Testing The latest main branch passes mvn -q clean test: 66 test classes, 339 test cases, 0 failures, 0 errors, and 0 skipped.

Functional Scope

We divide the functional scope into two categories according to actual usage scenarios: human-vs-machine play and online play. Both modes share the same backend rules and state-settlement logic. Their main differences lie in the source of player seats, connection method, and synchronization requirements.

Match Type	Main Features	Implementation Method
Human-vs-Machine	Start a 2-5 player game, human versus AI, AI difficulty selection, drawing cards, playing cards, rent settlement, action-card effects, win/loss detection, pause/resume, quit, wild-property recoloring, and save/load.	StartSessionRequest describes start parameters; SeatAssembler assembles human and AI players; TurnFlowService, ActionEffectDispatcher, RentCalculator, and PaymentSettlement jointly implement the game rules.
Online Match	Multiple clients join the same room, the host configures seats, room list, reconnection retention, real-time state synchronization, hand privacy protection, and multiplayer pause/save/load confirmation.	LobbyService manages rooms; SessionHub maintains sessions; MessageRouter routes WebSocket messages and prevents players from impersonating others; public STATE_UPDATE and private MY_HAND are sent separately.

Functional and Non-functional Boundaries

Implementation priority follows the principle of "core gameplay loop first, enhanced features later". The core loop includes starting a game, drawing cards, playing cards, resolving rent/action cards, turn progression, and victory detection. Enhanced capabilities include the multiplayer lobby, WebSocket privacy synchronization, AI difficulty, pause voting, quit handling, wild-property recoloring, save/load, reproducible seeds, and the auxiliary Web client.

Non-functional Feature	Design Handling
Usability	JavaFX uses card images, color labels, target-selection dialogs, rule descriptions, and Chinese/English resource files to reduce the learning difficulty.
Reliability	The server centrally validates all key actions. Illegal requests return errors while preserving consistency of the domain state.
Privacy	Public snapshots include only the number of other players' hand cards, while private hands are sent separately through bound player connections.
Maintainability	Responsibilities are split by packages such as controller, model, network, dto, persistence, pattern, fx, and presentation, with interfaces used to isolate changes.
Verifiability	Key rules, protocols, saves, AI, and performance smoke tests all have JUnit tests. Testing focuses on major business methods and boundary behavior.

2. Clean Design and Architecture

The system adopts a layered architecture: the client is responsible for display and input, the server is responsible for rules and state, DTOs define the protocol boundary, the domain model implements Monopoly Deal rules, and the persistence layer handles saving and restoring. In this way, the GUI, network protocol, and game rules can be modified and tested separately.

JavaFX Client -> WebSocket Endpoint -> MessageRouter / Dispatcher -> GameController Facade -> Services + Domain Model -> Snapshot + Broadcast

Layer/Package	Core Responsibility	Representative Classes
fx	JavaFX desktop GUI, card display, target selection, and WebSocket client. It is the main front-end interaction entry of the	MonopolyFxApp, MainController, CardView, PlayerBoardPanel

	project.	
network	Connection management, message routing, room lobby, player authorization, reconnection state, save/load voting, and state broadcasting.	GameServer, MessageRouter, LobbyService, SessionHub
controller	Facade and business-flow services, handling turns, effect stack, AI, pause/resume, quit, saves, and snapshots.	GameController, TurnFlowService, EffectStackOrchestrator, SnapshotBuilder
model	Cards, players, rules, settlement, effects, and the core game state.	Card, Player, ActionEffect, RentCalculator
dto	Data formats transferred through WebSocket and the game-state format sent from the server to clients.	PlayActionRequest, GameStateSnapshot, StartSessionRequest
persistence	Memento saves, persisted card snapshots, and optional AES-GCM encryption.	GameSessionMemento, PersistedCard, SaveEncryption
pattern	A central package for design-pattern examples used in the project.	CardFactory, GameUpdateSubject, AiPlayStrategy

Clean Design Explanation

In design, our focus is on component separation, small and clear class responsibilities, reasonable interfaces, correct data structures, restrained inheritance, and design patterns that serve real complexity. The latest main branch contains about 146 Java source files. Core logic is split by packages and services rather than being concentrated in one all-purpose class.

Clear Class and Module Responsibilities

The most important refactoring keeps GameController as the Facade while extracting responsibilities that were likely to grow too large. External calling entrances remain stable, while internal implementation is no longer carried by one class alone.

Responsibilities That Were Easy to Mix Before	Classes Now Taking Them	Design Benefit
Client-command validation and dispatch	ClientCommandHandler, MessageRouter	Request legality, player authorization, and business execution are separated, making error handling more centralized.
Assembly of public state and private hand snapshots	SnapshotBuilder	Broadcasting logic and game rules are separated, making the privacy boundary clearer.
New-session initialization, shuffling, and dealing	SessionFactory	The start flow is independent, making testing and reproducible experiments easier.
Construction of player, AI, and CUSTOM seats	SeatAssembler	Seat configuration is decoupled from turn rules, so adding new AI types has less impact.
Network room, connection, and voting collaboration	LobbyService, SessionHub, SaveLoadVoteCoordinator	The server entry no longer carries all lobby, reconnection, session, and voting details.
Turn state machine and play restrictions	TurnFlowService	The lifecycle of DRAW, PLAY, WAITING_FOR_RESPONSE, and END_TURN is explicit.
Rent response window and Just Say No chain	EffectStackOrchestrator	Complex response processes are separated from ordinary play logic.

Interface Usage

Interfaces are used to express object capabilities that have similar behavior but are not suitable for the same inheritance tree, rather than adding layers for formality.

Interface	Usage Scenario	Why It Is Suitable
Playable	Whether a card can be played in the current context.	Different card types have different playability rules, but they all need one unified validation entry.
Payable	Money cards and action cards stored in the bank can both be used for payment.	Payability is a capability, so all payment objects should not be forced to share a complex parent class.
ActionEffect	Execution strategy for all action-card effects.	Adding a new action-card effect means adding one implementation class instead of changing large conditional branches.
AIPlayStrategy	AI decisions such as Easy, Normal, Hard, Lookahead, and Local Ranker.	AIPlayer composes strategy objects, making difficulty and experimental strategies easy to replace.
GameUpdateSubject / GameUpdateObserver	State-update publication and network broadcast.	The controller does not need to know how many display ends or connection ends exist.
ClientConnection	Abstraction of a WebSocket connection.	The network layer can replace the connection implementation during tests.
AIGameBridge	AI reuses the same rule entry as human players.	This prevents AI from bypassing server validation and reduces rule branching.

Data Structure Choices

Data Structure	Usage Location	Reason
List<Card>	Hand, draw pile, discard pile, bank area, property area, and action area.	Card areas need to preserve order and support lookup by index or id.
Player list + current index	TurnManager	Turn order advances in a cycle, and a list with an index expresses this directly.
Map / bidirectional indexes	SessionRegistry, SessionHub	Multiplayer connections need accurate routing among session, connection, and playerId.
Independent deck per session	Engine instance held by GameController	Each session owns its own draw pile, discard pile, shuffle, and card-count statistics, avoiding shared deck state across games.
enum	Limited states such as turn phase, building level, and wild-property type.	This reduces string magic values and improves compile-time checkability.
Effect-stack objects	EffectStackEntry, StackResponseState	Rent, Double Rent, and Just Say No form a chained response. Object representation is clearer than scattered if-else logic.
DTO objects	PlayActionRequest, GameStateSnapshot	Protocol data is separated from domain objects, preventing business logic from depending directly on raw JSON.

Inheritance Relationships

Inheritance is used only for natural is-a relationships. Card is an abstract card; PropertyCard, PropertyWildCard, ActionCard, and MoneyCard represent different card types. Player is an abstract player; HumanPlayer and AIPlayer represent human players and computer players. For cross-layer behaviors such as payable, playable, observable, and decidable, the project prefers interfaces and composition to avoid overly deep inheritance hierarchies.

Design Pattern Applications

All design patterns can be located in the code and serve real problems.

Design Pattern	Code Location	Role
MVC	model, controller, fx/MainController, FXML	Separates the rule model, control flow, and GUI

		presentation.
Facade	GameController	Provides a unified entry to the GUI, network layer, and tests while hiding internal service collaboration details.
Observer	GameUpdateSubject, GameUpdateObserver, DefaultGameUpdateSubject	Notifies the network layer to broadcast state when the game state changes.
Factory Method	CardFactory, MonopolyDealCardFactory	Uniformly creates the standard 108-card Monopoly Deal deck and ensures consistent card types, counts, and rent-card configuration.
Strategy	AiPlayStrategy and multiple AI implementations	Extracts AI decision algorithms from the player entity, making it easy to choose Easy, Normal, Hard, Lookahead, and other difficulties.
Strategy	ActionEffect and each action-card effect class	Encapsulates different action-card effects as independent strategies, avoiding one huge switch for all rules.
Memento	GameSessionMemento, SessionPlayerMemento, PersistedCard	Captures and restores the full game state to implement saving, loading, and auto-saving.

3. Core Feature Implementation

Feature implementation revolves around a complete game loop while avoiding class-level detail overload. Overall, the project has formed a runnable Monopoly Deal loop: creating a game, player turns, card rules, settlement, state synchronization, victory, and saving.

Game Modes and Start Flow

- Supports 2-5 players. Start parameters include sessionId, player count, game mode, AI difficulty, random first-player setting, and custom lineup.
- In HVM mode, human players play against AI. AI supports Easy, Normal, Hard, and stronger local strategies through AiPlayStrategy.
- In PVP/multiplayer mode, multiple clients join the same session through WebSocket. The lobby service supports room creation, joining rooms, room list, automatic seating, seat setting, room starting, and retaining reconnection information after the game starts.
- MonopolyDealCardFactory generates the standard 108-card deck, including properties, wild properties, cash, rent cards, and action cards.

Turn Flow

1. Draw phase: under normal conditions, draw 2 cards; if the hand is empty before the turn starts, draw 5 cards according to the rules.
2. Play phase: each turn allows at most 3 play actions, including banking a card, deploying a property, and executing an action card.
3. Response phase: rent, debt collection, card stealing, and similar effects enter a waiting response window. The target player can use Just Say No or choose to pay/decline a response.
4. Discard and end turn: when the hand exceeds 7 cards, the player must discard. After the limit is satisfied, the turn moves to the next player.

TurnFlowService explicitly maintains turn phases, so the client cannot bypass rules at the wrong time. For example, while waiting for a response, only response-related actions are allowed, and ordinary card play is rejected.

Cards, Rent, and Action Cards

Feature Area	Implementation Description
Properties and Wild Properties	Supports deploying properties by color. Wild properties can be deployed according to allowed colors and can be recolored during the player's turn through REASSIGN_WILD. Complete sets are determined by PropertySetCalculator.

Cash and Bank	Money cards and action cards already stored in the bank can be used as payment assets. After an action card is banked, only its monetary value remains.
Rent	RentCalculator calculates rent by color, set progress, house, and hotel. Rent tiers are centralized in RentTierTable.
Payment	PaymentSettlement forbids payment from hand cards. Payment can only use assets from the table bank/property area, and overpayment gives no change.
Action Cards	Implements core effects such as Pass Go, Rent, Double Rent, Debt Collector, Birthday, Sly Deal, Forced Deal, Deal Breaker, House, Hotel, and Just Say No.

Victory Detection

The system continuously monitors each player's property area. When a player reaches three complete property sets, the system generates a victory state and ends the match. Victory detection is centralized in the backend rule layer, so the GUI and AI reuse the same logic and avoid inconsistent results caused by separate calculations on different ends.

Pause, Quit, and State Prompts

The project adds pause state to GameStateSnapshot, including paused, pausePending, pauseAckCount, and pauseHumanCount. HVM mode can pause/resume directly; PVP mode keeps multiplayer state consistent through a confirmation flow. Quit logic also distinguishes HVM from PVP: in a human-vs-machine match, a human quit forcibly ends the current game, while in a multiplayer match, a single human quit does not directly end the whole game.

GUI and User Experience

The JavaFX client is the main interactive interface of the project. The interface provides start/lobby screens, hand cards, player boards, card images, target selection, wild-property recolor buttons, pause/quit menus, rule descriptions, operation buttons, and state prompts. The GUI does not directly modify domain objects. Instead, it requests backend execution through JSON messages, and the backend returns the latest snapshot to refresh the interface. In this way, the server is always the rule authority, which is especially suitable for multiplayer mode.

The Web/Vite side is retained as an auxiliary entry, mainly for testing and covering more scenarios, such as quickly verifying room lists, different CUSTOM seat combinations, AI/human mixed seats, and multiplayer state synchronization. It does not replace the main JavaFX GUI deliverable; it helps cover more paths during development and pre-submission checks.

Multiplayer Synchronization and Privacy Protection

The WebSocket protocol uses a unified JSON envelope. Public message STATE_UPDATE contains only public information, such as player assets, total bank value, hand-card counts, current turn, and deck counts. Private message MY_HAND is sent only to the corresponding player connection. This design allows players to see the public board while preventing them from seeing other players' hands through broadcast messages.

Save, Load, and Encryption

GameSessionMemento captures the draw pile, discard pile, player hands, bank areas, property areas, action areas, current turn, response window, and effect stack. SaveLoadService supports manual saving, loading, and auto-saving every 3 full rounds. After monopoly.saveKey is set, SaveEncryption uses AES-GCM to protect local saves; without the setting, plaintext JSON is retained for debugging.

4. Innovation: DeepSeek Integration, Local Model Training, and Online Capability

DeepSeek and the Local Student Model

In addition to the basic Easy, Normal, and Hard AI, this project implements a more exploratory AI flow: first, it integrates DeepSeek as an online strong strategy, letting the model choose from legal candidate actions generated by the server;

then it uses these decision data and local search/heuristic results to train a small local model, enabling the project to run a "student model" AI without calling a remote LLM.

AI Entry	Code/Resource Location	Role
DeepSeek online strategy	DeepSeekAiPlayStrategy, DeepSeekClient	The server first enumerates legal candidate actions, then asks DeepSeek to return only the candidate id, preventing the model from inventing illegal operations.
Search-enhanced local AI	SearchLookaheadAiPlayStrategy	Simulates candidate actions in an isolated session and ranks them by indicators such as property sets, opponent threats, and rentbenefit.
Local student model	LocalRankerAiPlayStrategy, backend/models/distillation/	Loads a trained linear/small MLP ranker JSON and uses a lightweight model to score legal candidates.
Mode exposure	SeatAssembler, JavaFX/Web resources	LLM uses DeepSeek; student uses the local student model, and falls back to search-enhanced AI if model loading fails.

The key point of this design is that the LLM does not directly control the game state. AI can only choose from backend-validated candidate actions. The real rule validation and state modification are still completed by TurnFlowService, ActionEffect, and settlement services. Therefore, it demonstrates innovation without breaking the rule-authority boundary in clean design.

Training artifacts are stored in backend/models/distillation/, such as lookahead-student-v13-actiongateA-dagger3125-w3-listwise-mlp and deepseek-production-20260524. We did not only write a fixed heuristic AI; we also tried to use DeepSeek to generate decision signals and distill them into a small model that can run locally.

Online Rooms and Multi-client Synchronization

The online part is not just a simple connection from clients to the server. It is built around real multiplayer board-game needs, including rooms, seats, privacy, authorization, reconnection, and voting flows. Players can create rooms, join rooms, let the host set human/AI/LLM/student seats, and then start the game together. It also includes a room list, duplicate-nickname rejection, connection-count statistics for started rooms, and authorization checks that bind client actions to the correct player. This design allows the same rule set to support both local human-vs-machine games and multiplayer online or mixed-AI testing.

Online Capability	Implementation Location	Value
Rooms and seats	LobbyService, SeatAssembler	Supports creating/joining rooms, room lists, automatic seating, and configuring human players with different AI types, making it easier to demonstrate multiple match combinations.
Session isolation and reconnection	SessionHub, SessionRegistry	Players, connections, and states from different sessions do not interfere with one another. After a room has started, disconnection does not directly delete room-member information.
Privacy synchronization	SnapshotBuilder, MY_HAND, STATE_UPDATE	Public state and private hand cards are sent separately, preventing players from seeing other players' hands.
Authorization and multiplayer confirmation	MessageRouter, PauseVoteService, SaveLoadVoteCoordinator	Actions such as playing cards, drawing cards, ending turns, and confirming save/load are validated according to the connection-bound player. Pause, save, and load use multiplayer confirmation to maintain consistency.

5. Testing and Validation

Testing focuses on major functions and key methods rather than simple getter/setter methods. This project uses JUnit 5 and Maven, with emphasis on rule boundaries, service flows, protocol contracts, save restoration, and performance smoke tests.

Validation Item	Command	Latest Result
Complete backend test	mvn -q clean test	Passed: 66 test classes, 339 test cases, 0 failures, 0 errors, 0 skipped.
Web/Vite auxiliary-side build	npm run build --prefix frontend	Passed: confirms that the auxiliary testing entry can build normally.

Testing Area	Representative Test Classes	Coverage Focus
Deck and rules	DeckCompositionTest, DeckShuffleOrderTest, MonopolyDealBankValuesTest	Standard 108-card deck, shuffle order, and bank values of action cards.
Turn flow	OverflowDiscardRuleTest, DoubleRentNextRentTest, DeckBoundaryRuleTest, HvmQuitEndsGameTest	Discard limit, Double Rent effect, deck boundary, and HVM/PVP quit behavior.
Action-card effects	DealBreakerEffectTest, ForcedDealEffectTest, StealCardEffectTest, HouseHotelEffectTest	Core effects such as stealing, exchanging, building upgrades, and taking a complete set.
Rent and payment	RentCalculatorPartialSetTest, RentCalculatorBuildingBonusTest, PaymentSettlementExplicitTest	Partial-set rent, building bonus, explicit payment, and overpayment without change.
Save and encryption	GameSessionMementoTest, SaveEncryptionTest, SaveLoadIntegrationTest	Full-state capture/restoration, AES-GCM encryption, and load integration.
Network protocol	PingProtocolTest, PlayProtocolValidationTest, GameServerEndToEndSmokeTest	Message parsing, error codes, and end-to-end session.
Multiplayer mode	GameServerMultiSessionIsolationTest, GameServerPvpSaveVoteTest, GameServerPvpLoadVoteTest, GameServerRoomListTest, GameServerAuthorizationGuardTest	Session isolation, room list, nickname conflicts, multiplayer save/load voting, connection-bound player authorization, and reconnection state.
AI and performance	AiStrategyProfileDifferentiationTest, SearchLookaheadAiPlayStrategyTest, PerformanceSmokeTest	AI difficulty differences, strong-strategy behavior, and basic performance-regression checks.

6. Teamwork and Member Contributions

The teamwork section mainly explains how we divided work and how we used GitHub to record the process. The project commit history covers requirements organization, UML, core rules, JavaFX, WebSocket, AI, testing, documentation, and final refactoring.

Member Contribution Table

Member	Student No. / GitHub	Responsibility / Share	Contribution Description
Liu Zongrun	24107745 LiuZongrun7	AI model development and training 20%	Responsible for DeepSeek integration, the local student-model approach, AI decision-candidate design, training-artifact organization, and experimental validation of different AI strategies.
Lei Yurun	24107684 NEVER-AGAIN-RAY	Backend development 20%	Responsible for UML diagram design, backend core logic, WebSocket message handling, turn flow, rule settlement, service splitting, and backend refactoring.

Member	Student No. / GitHub	Responsibility / Share	Contribution Description
Wang Tingdong	24107759 Harry-magic	Frontend development 20%	Responsible for requirements-document writing, JavaFX + FXML desktop-client development, card/player-board display, interaction flow, target selection, and front-backend integration; also helped maintain the Web side as an auxiliary testing entry.
Liu Yuhao	24107764 YuhaoIIIIII	Testing and quality checks 20%	Responsible for functional testing, rule-boundary validation, regression-test organization, runtime compatibility checks, and pre-submission quality confirmation.
Qiu Siqu	24107750 qiusiqu-2024	Documentation and review 20%	Responsible for requirements-document organization, submission-material checking, presentation-content preparation, final-report review, and assisting with multiplayer-flow validation.